

SENTIMENT DATASET

The FinBERT-based approach focused on enhancing text representation by generating finance-specific sentiment features from the `full_text` column and combining these with the existing sentiment variables (`sentiment_pos`, `sentiment_neu`, `sentiment_neg`, and `sentiment_polarity`) to predict next-period stock price direction. Rather than relying on TF-IDF representations, FinBERT was used to extract structured sentiment probabilities (positive, negative, and neutral), along with derived features such as polarity, which were then used as inputs into a logistic regression classifier. This approach leverages domain-specific language understanding, allowing the model to better capture financial tone and context embedded in news text while maintaining a lightweight and CPU-friendly modeling pipeline.

The baseline logistic regression model using FinBERT-derived features achieved an accuracy of 55.78%, representing an improvement over earlier TF-IDF-based models. However, performance was uneven across classes. The model demonstrated strong ability to identify upward movements, with a recall of 0.82 and F1-score of 0.66, but struggled significantly with downward movements, achieving a recall of only 0.26. This indicates a bias toward predicting the “Up” class, which is further reflected in the balanced precision across classes (0.56) but weaker macro-averaged F1-score (0.51). After applying GridSearch hyperparameter tuning, the model’s accuracy slightly decreased to 54.18%, and the imbalance became more pronounced. The tuned model achieved an even higher recall for the “Up” class (0.88) but a much lower recall for the “Down” class (0.16), suggesting that the optimization process reinforced the model’s tendency to favor the majority class. Overall, while FinBERT features improved the model’s ability to capture positive market signals, the results highlight persistent challenges with class imbalance and limited discriminative power for downward movements, indicating that further feature engineering, rebalancing techniques, or alternative modeling approaches may be necessary to achieve more balanced predictive performance.

The grid search process evaluated 64 hyperparameter combinations across 5 folds, resulting in a total of 320 model fits, which provides a robust assessment of model performance across different regularization settings. The best-performing configuration was a logistic regression model with elastic net regularization (`penalty = 'elasticnet'`) using the saga solver, a regularization strength of $C = 0.1$, and an `l1_ratio` of 0.5, indicating an equal balance between L1 (feature selection) and L2 (shrinkage) penalties. This suggests that the model benefits from both reducing coefficient magnitude and selectively zeroing out less important features, which is particularly useful when working with a mix of sentiment-based features and potentially noisy signals derived from text.

An important observation is that the mean test score (0.692) and mean train score (0.690) for the best model are very close, indicating minimal overfitting and good generalization during cross-validation. Additionally, several top-performing configurations produced nearly identical scores, particularly those with stronger regularization ($C = 0.01$ or 0.1) and variations of L1 or elastic net penalties. This consistency suggests that the model’s performance is relatively stable across different regularization settings, but also indicates that hyperparameter tuning alone may not significantly improve predictive performance. Despite strong cross-validation results, the drop in validation performance seen earlier implies a potential distribution mismatch between training folds and the validation set or that the model is

still biased toward the majority class. Overall, these findings reinforce that while regularization helps control model complexity and improve stability, further gains will likely come from better feature engineering or class imbalance handling rather than additional tuning of the same model family.

COMBINED DATASET

In week 8, we decided to implement another complex model with a tree-based algorithm. XGBoost was a logical choice, as this model builds on the basic decision tree by implementing boosting, a method that uses each prior decision tree to inform the next. The model also uses gradient descent, which attempts to minimize the loss in the model's performance over the iterations. This gives it similar attractive properties as a neural network, but it requires less data to train on, which is attractive for our project since we only have approximately two thousand observations in the combined dataset.

After implementing an initial XGBoost model, three hyper-parameters were tuned using f1 score as the main scoring metric. Hyper-parameters included:

- **N_estimators:** Represents the number of individual sequential trees used to classify stock price direction. One of [100, 400, 750]
- **Max_depth:** The maximum depth of each individual tree. One of [3, 4, 5]
- **Learning_rate:** The contribution rate of each new tree to the overall model. This helps prevent overfitting. One of [0.01, 0.02, 0.05, 0.10]

The initial model with default parameters performed at 52% overall accuracy and a 0.55 F1 score for the “up” stock price direction class. When using 5-fold cross-validation grid search across the hyper-parameters listed above, f1 score was used as the primary scoring metric while accuracy was used as a secondary scoring metric. The following results were obtained:

Number of Trees	Max Depth	Learning Rate	Training Accuracy	Training F1	Test Accuracy	Test F1
100	3	0.01	100%	1.00	100%	1.00
400	3	0.01	100%	1.00	100%	1.00
700	3	0.01	100%	1.00	100%	1.00
100	4	0.01	100%	1.00	100%	1.00
400	4	0.01	100%	1.00	100%	1.00
700	4	0.01	100%	1.00	100%	1.00
100	5	0.01	100%	1.00	100%	1.00
400	5	0.01	100%	1.00	100%	1.00

Group Forecast and Furious
Casey Barrasso, Joshua Previte, Giovanni Jones

700	5	0.01	100%	1.00	100%	1.00
-----	---	------	------	------	------	------

There were many grid searches that did perfectly on each individual cross-section. When combined, the model looked to do perfectly across the entire dataset. This is likely because the training dataset had many observations that were in the “up” stock price direction (this was when the AAPL stock price accelerated), while the validation set is much more mixed. However, when tested across the entire validation set at once, accuracy was 57% while the F1 score on the “up” class was 0.71. This is an improvement upon the initial XGBoost model as well as the support vector classifier from last week. This XGBoost model is now the leading model in performance.

HISTORICAL PRICE DATASET

We also used an XGBoost model as our third model on the historical price dataset. Since logistic regression and SVM both performed poorly with the historical price data, it was a logical next step to see if a tree-based model would make a difference. The initial XGBoost model, however, also performed poorly with the price data with an accuracy of 45%.

To examine the effects of hyperparameter tuning on the dataset, a grid search was performed using the same combinations of parameters as the combined dataset. A sample of the outcome is listed in the following table:

Number of Trees	Max Depth	Learning Rate	Training Accuracy	Training F1	Test Accuracy	Test F1
750	5	0.01	0.5127	0.4803	0.5497	0.5212
400	5	0.01	0.5127	0.4803	0.5497	0.5212
750	4	0.01	0.5127	0.4803	0.5497	0.5212
400	4	0.01	0.5127	0.4803	0.5497	0.5212
400	5	0.1	0.5127	0.4803	0.5497	0.5212
750	5	0.1	0.5127	0.4803	0.5497	0.5212
100	5	0.1	0.5127	0.4803	0.5497	0.5212
750	4	0.1	0.5127	0.4803	0.5497	0.5212
100	5	0.02	0.5127	0.4803	0.5497	0.5212
750	4	0.02	0.5127	0.4803	0.5497	0.5212
400	4	0.02	0.5127	0.4803	0.5497	0.5212
100	4	0.02	0.5127	0.4803	0.5497	0.5212

Group Forecast and Furious
Casey Barrasso, Joshua Previte, Giovanni Jones

400	5	0.02	0.5127	0.4803	0.5497	0.5212
750	5	0.02	0.5127	0.4803	0.5497	0.5212
400	5	0.05	0.5127	0.4803	0.5497	0.5212
750	5	0.05	0.5127	0.4803	0.5497	0.5212
100	5	0.05	0.5127	0.4803	0.5497	0.5212
750	4	0.05	0.5127	0.4803	0.5497	0.5212
400	4	0.05	0.5127	0.4803	0.5497	0.5212
100	4	0.05	0.5127	0.4803	0.5497	0.5212

The results indicate that hyperparameter tuning provides little improvement in the performance of the XGBoost model. Since none of the models performed well using the historical price dataset, it is likely that the dataset lacks sufficient predictive information. The feature set is relatively small and simplistic, limiting its ability to effectively predict stock price direction.